DocMind to Production
Into the Cloud

Notes – 3
AWS · ECR · GitHub Actions CI Pipeline

Saturday, April 4

Previously, DocMind moved into a Docker container. Today that container moves into AWS. The Docker image lives in the cloud and every push to GitHub triggers an automated build.

PART 1	PART 2	PART 3
AWS Fundamentals	**ECR · ECS · Docker**	**GitHub Actions CI**
Accounts · IAM · Regions	Push images to the cloud	Automate every code push

Part 1: AWS — Understanding the Cloud

1.1 What is AWS and Why Does It Matter?

AWS (Amazon Web Services) is a massive collection of computing services that Amazon rents out over the internet. Instead of buying a server and managing it yourself, you rent exactly what you need, pay per second, and let Amazon handle the hardware.

> **The Office Park Analogy**
>
> Imagine you want to run a business. You could buy land, construct a building, install electricity, hire security guards, manage maintenance — or you could rent an office in a commercial park where all of that is already done.
>
> AWS is the world's largest commercial office park for software. You rent exactly the space you need. Amazon handles the building, the power, the security. You just run your business.

For DocMind, AWS gives you:

- **A place to run your Docker container** (ECS — Elastic Container Service)
- **A place to store your Docker images** (ECR — Elastic Container Registry)
- **A place to store uploaded PDFs** (S3 — Simple Storage Service)
- **A way to monitor everything** (CloudWatch)
- **A front door for your API** (API Gateway and Load Balancer)

1.2 Creating Your AWS Account

AWS offers a **free tier** — a set of services you can use for free for 12 months (with usage limits). Everything you do today fits within the free tier.

> **Before you start**
>
> You need: a valid email address, a credit/debit card (required for identity verification, you won't be charged if you stay within free tier limits), and a phone number for verification.
>
> URL: aws.amazon.com — click 'Create an AWS Account'. The sign-up takes about 10 minutes.

1	**Create your AWS account** 1. Go to aws.amazon.com and click Create an AWS Account 2. Enter your email and choose a root account password 3. Choose Personal account type 4. Enter billing information (card required but free tier won't charge you)

5. Verify your identity via phone
6. Choose the Free tier support plan
7. Wait for the confirmation email (can take a few minutes)

Important: Never use the root account for daily work

The root account is like the master key to your entire AWS account. If someone gets access to it, they can do anything — spin up expensive servers, access all your data, delete everything.

AWS best practice: create an IAM user for daily work and never log in as root again (except for billing).

1.3 IAM — Identity and Access Management

IAM is AWS's security system. It controls **who** can do **what** in your AWS account. Every action in AWS is a permission that must be explicitly granted.

The Hotel Key Card Analogy

A hotel has many rooms. The master key (root account) opens everything. But the room key (IAM user) only opens specific rooms the hotel has assigned.

A cleaner gets a key that opens rooms but not the safe. A guest gets a key for one room only. IAM works the same way — every identity gets exactly the permissions it needs and nothing more.

This principle is called Least Privilege: give everyone the minimum permissions they need to do their job.

Creating an IAM user for daily work

1 **Create an IAM user**

8. In the AWS Console, search for IAM in the top search bar
9. Click Users in the left sidebar, then Create user
10. Username: docmind-dev
11. Check: Provide user access to the AWS Management Console
12. Choose: I want to create an IAM user
13. Set a password
14. Click Next: Permissions

2 **Attach permissions**

For today, attach these managed policies (search for each by name):

- **AmazonEC2ContainerRegistryFullAccess** — lets you push images to ECR
- **AmazonECS_FullAccess** — lets you deploy to ECS
- **AmazonS3FullAccess** — lets you manage S3 buckets
- **CloudWatchFullAccess** — lets you view logs and metrics

Click Next, then Create user. Save the login URL, username, and password.

Creating Access Keys (for CLI and GitHub Actions)

The IAM user you just created can log into the web console. But GitHub Actions and the AWS CLI need a different kind of credential called an **Access Key** — a pair of tokens (Access Key ID + Secret Access Key) that programs use to authenticate with AWS.

3

Create access keys

15. Click on your new IAM user (docmind-dev)
16. Go to the Security credentials tab
17. Click Create access key
18. Select: Command Line Interface (CLI)
19. Click Next, then Create access key
20. **CRITICAL:** Download the .csv file immediately. The Secret Access Key is shown ONCE and never again. If you lose it, you must create a new key pair.

1.4 AWS Regions — Where Your Infrastructure Lives

AWS has data centres all over the world, grouped into **regions**. A region is a geographic area with multiple data centres. When you create any AWS resource, you choose which region it lives in.

Region	Code	Best for
Mumbai (closest to Bangalore)	ap-south-1	Lowest latency for Indian users. Good default for your projects.
N. Virginia (US East)	us-east-1	AWS default. Most services available here first.
Ireland (EU West)	eu-west-1	Good for EU users — relevant for your Amsterdam goal.
Singapore	ap-southeast-1	Alternative Asia-Pacific option.

> **Use ap-south-1 (Mumbai) for everything today**
>
> Resources in different regions can't automatically talk to each other. If you create your ECR in Mumbai and accidentally create ECS in Singapore, your deployment won't work.
>
> Pick one region and stick to it for every resource you create today. Mumbai (ap-south-1) makes sense given you're in Bangalore.

1.5 AWS CLI — Controlling AWS from Your Terminal

The AWS Console is a web interface for humans. The **AWS CLI** (Command Line Interface) is a tool for controlling AWS from your terminal — same thing, but faster, scriptable, and what your CI/CD pipeline uses.

1

Install the AWS CLI

macOS:

```
curl "https://awscli.amazonaws.com/AWSCLIV2.pkg" -o "AWSCLIV2.pkg"
sudo installer -pkg AWSCLIV2.pkg -target /
```

Ubuntu/Linux:

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install
```

Verify installation:

```
aws --version
# Expected: aws-cli/2.x.x Python/3.x.x ...
```

2

Configure the CLI with your IAM credentials

```
aws configure
```

You'll be prompted for four things:

- **AWS Access Key ID:** paste from your .csv file
- **AWS Secret Access Key:** paste from your .csv file
- **Default region:** ap-south-1
- **Default output format:** json

Verify it worked:

```
aws sts get-caller-identity

# Expected output:
# {
#   "UserId": "AIDAIOSFODNN7EXAMPLE",
#   "Account": "123456789012",
#   "Arn": "arn:aws:iam::123456789012:user/docmind-dev"
# }
```

Part 2: ECR — Pushing DocMind into the Cloud

2.1 What is ECR?

ECR (Elastic Container Registry) is AWS's service for storing Docker images. Think of it as DockerHub, but private and living inside your AWS account. Your ECS cluster pulls images from ECR to run containers.

> **The Shipping Container Yard Analogy**
>
> Yesterday you vacuum-sealed your kitchen into a Docker image (the container). Now you need to ship it somewhere AWS can pick it up.
>
> ECR is the container yard at the port. You drive your container there (docker push), leave it, and AWS (the shipping company) picks it up and deploys it wherever you need.

The full flow from code to running app:

Step	What happens	Where
1. Code change	You edit main.py or add a feature	Your laptop
2. docker build	Docker packages code into an image	Your laptop
3. docker push	Image uploaded to ECR	AWS cloud
4. ECS pulls image	ECS downloads the image from ECR	AWS cloud
5. Container runs	Your app is live and serving users	AWS cloud

2.2 Creating an ECR Repository

A repository in ECR is like a folder that holds all versions (tags) of one image. You'll create one repository called **docmind** and push different versions to it: **docmind:v1**, **docmind:v2**, **docmind:latest**.

1

Create the repository via AWS Console

21. In the AWS Console, search for ECR in the top search bar
22. Make sure you're in ap-south-1 (Mumbai) — check the region dropdown top-right

23. Click Create repository
24. Repository name: docmind
25. Keep all defaults (Private visibility, tag mutability: Mutable)
26. Click Create repository
27. Click on the docmind repository — copy the URI shown at the top

The URI looks like:

```
123456789012.dkr.ecr.ap-south-1.amazonaws.com/docmind

# Structure:
# <account-id>.dkr.ecr.<region>.amazonaws.com/<repo-name>
```

Save this URI. You'll use it in every push command.

2

Create the repository via CLI (alternative)

```
aws ecr create-repository \
  --repository-name docmind \
  --region ap-south-1

# Save the repositoryUri from the output
```

2.3 Authenticating Docker to ECR

Before you can push images to ECR, Docker on your laptop needs to authenticate with AWS. ECR uses temporary tokens that expire after 12 hours — you run one command to get a fresh token and pass it to Docker.

Why authentication is needed

Your ECR repository is private. Only people (or systems) with the right AWS credentials can push to it. The authentication command gets a temporary password from AWS and tells your local Docker to use it.

Think of it as signing in to a private warehouse before you can drop off goods.

1

Authenticate Docker to ECR

```
# Replace 123456789012 with your AWS account ID
# Replace ap-south-1 with your region if different
aws ecr get-login-password --region ap-south-1 \
```

```
  | docker login --username AWS \
  --password-stdin \
  123456789012.dkr.ecr.ap-south-1.amazonaws.com

 # Expected output:
 # Login Succeeded
```

To find your account ID:

```
 aws sts get-caller-identity --query Account --output text
```

2.4 Tagging and Pushing Your Image

Docker images need to be **tagged** with the full ECR URI before they can be pushed. Tagging is just adding an alias to an existing image — it doesn't create a new image, it just gives the existing one a new name.

1

Tag your local image with the ECR URI

```
 # Format: docker tag <local-image>:<tag> <ecr-uri>:<tag>
 docker tag docmind:v1 \
  123456789012.dkr.ecr.ap-south-1.amazonaws.com/docmind:v1

 # Also tag as 'latest' (convention: latest = most recent stable)
 docker tag docmind:v1 \
  123456789012.dkr.ecr.ap-south-1.amazonaws.com/docmind:latest

 # Verify both tags exist
 docker images | grep docmind
```

2

Push the image to ECR

```
 # Push the versioned tag
 docker push 123456789012.dkr.ecr.ap-south-1.amazonaws.com/docmind:v1

 # Push the latest tag
 docker push 123456789012.dkr.ecr.ap-south-1.amazonaws.com/docmind:latest

 # Expected output (each layer uploads separately):
 # v1: digest: sha256:abc123... size: 312345678
```

3

Verify the image is in ECR

```
aws ecr list-images --repository-name docmind --region ap-south-1

# Expected:
# {
#   "imageIds": [
#     { "imageDigest": "sha256:...", "imageTag": "v1" },
#     { "imageDigest": "sha256:...", "imageTag": "latest" }
#   ]
# }
```

Or check the AWS Console — go to ECR → docmind repository → you should see your image listed with its tags and size.

2.5 What is ECS? (Context for Later)

ECS (Elastic Container Service) is AWS's service for **running** Docker containers at scale. If ECR is the container yard where images are stored, ECS is the factory floor where containers are actually running and serving users.

Concept	Analogy	Technical meaning
Task Definition	Blueprint/recipe	Tells ECS which image to run, how much CPU/RAM, what ports, what env vars
Task	One running unit	A single container instance running from a Task Definition
Service	Factory floor manager	Keeps N tasks running at all times. If one crashes, starts a replacement.
Cluster	The factory building	Groups of compute resources where tasks run. Uses Fargate (serverless) or EC2.
Fargate	Serverless compute	AWS manages the servers. You just define CPU/RAM needs and pay per second.

Today vs Day 9

Today you push your image to ECR and set up automated builds. The actual ECS deployment (Task Definition, Service, Cluster) happens on Day 9 when you deploy to production.

Think of today as stocking the warehouse. Day 9 is opening the factory and starting production.

Part 3: GitHub Actions — CI Pipeline

3.1 What is CI/CD?

CI stands for Continuous Integration. **CD** stands for Continuous Deployment. Together they form an automated pipeline that takes your code from a **git push** to a running application without you touching anything manually.

> **The Car Factory Analogy**
>
> Imagine a car factory where a worker manually assembled every car by hand: weld the frame, install the engine, test it, paint it, inspect it, ship it. Slow, error-prone, inconsistent.
>
> A modern factory has a robotic assembly line. A component goes in one end, a finished tested car comes out the other. Every car identical. Much faster. No manual steps to forget.
>
> GitHub Actions is your assembly line for software. Code goes in (git push), tested built deployed app comes out. Automatically.

Term	Meaning	DocMind example
CI (Continuous Integration)	Automatically test and build on every push	Every push to main triggers docker build and tests
CD (Continuous Deployment)	Automatically deploy passing builds	After build succeeds, push image to ECR automatically
Pipeline	The full automated sequence	push → build → test → push to ECR → deploy to ECS
Trigger	What starts the pipeline	Pushing code to the main branch
Job	A group of steps	build-and-push is one job with 6 steps
Runner	The machine that runs your pipeline	GitHub provides virtual Ubuntu machines for free

3.2 GitHub Actions — How It Works

GitHub Actions reads workflow files from a special folder in your repository: **.github/workflows/**. Every **.yml** file in that folder is a workflow — a set of automated tasks.

> **Why YAML?**
>
> YAML is a human-readable data format — think of it as a very structured shopping list. Each item has a name and value. Indentation defines structure (nesting). GitHub Actions workflow files are written in YAML because they're easy to read and write.
>
> Critical rule: YAML uses spaces, not tabs. A tab character in a YAML file breaks the entire file with a cryptic error. Always use spaces.

Anatomy of a workflow file

A GitHub Actions workflow has five key sections:

Section	Purpose	Example
name	Human-readable name shown in GitHub UI	name: CI — Build and Push
on	What triggers the workflow	on: push: branches: [main]
env	Environment variables for all jobs	env: AWS_REGION: ap-south-1
jobs	The actual work to do	jobs: build-and-push: ...
steps	Individual tasks within a job	steps: - name: Build Docker image

3.3 GitHub Actions Secrets — Keeping Credentials Safe

Your CI pipeline needs AWS credentials to push to ECR. But you can't put them in the workflow file — that file is committed to GitHub. Anyone who sees your repository would see your credentials.

GitHub Secrets is the solution. Secrets are encrypted values stored by GitHub that your workflow can access at runtime as environment variables, but that are never visible to anyone reading the code.

> **How GitHub Secrets work**
>
> You store: AWS_ACCESS_KEY_ID = AKIAIOSFODNN7EXAMPLE in GitHub's encrypted vault.
>
> In your workflow you reference it as: ${{ secrets.AWS_ACCESS_KEY_ID }}
>
> GitHub injects the real value at runtime. In workflow logs, secrets are always shown as ***. Nobody can see the actual value, not even repository admins.

1

Add AWS credentials to GitHub Secrets

28. Go to your DocMind GitHub repository
29. Click Settings (top menu, not your profile settings)
30. Click Secrets and variables → Actions in the left sidebar
31. Click New repository secret for each of the following:

Secret name	Value to paste	Where to find it

AWS_ACCESS_KEY_ID	AKIA...	Your IAM credentials .csv file
AWS_SECRET_ACCESS_KEY	wJalr...	Your IAM credentials .csv file
AWS_ACCOUNT_ID	123456789012	aws sts get-caller-identity output

After adding all three, you should see them listed under Repository secrets (values are hidden).

3.4 Writing the CI Workflow File

Now write the actual workflow. This file goes at: **.github/workflows/ci.yml**

```
# .github/workflows/ci.yml
# This workflow runs every time code is pushed to the main branch.
# It builds the Docker image and pushes it to AWS ECR.

name: CI — Build and Push to ECR

# TRIGGER: What causes this workflow to run?
# on: push to main branch, OR manual trigger from GitHub UI
on:
  push:
    branches:
      - main
  workflow_dispatch:    # Adds a 'Run workflow' button in GitHub Actions UI

# ENVIRONMENT VARIABLES available to all jobs
env:
  AWS_REGION: ap-south-1
  ECR_REPOSITORY: docmind

# JOBS: the actual work
jobs:
  build-and-push:

    # The machine GitHub provides to run this job
    # ubuntu-latest = a fresh Ubuntu VM, starts clean every run
    runs-on: ubuntu-latest

    steps:

      # Step 1: Download your code onto the runner machine
      # 'actions/checkout@v4' is a pre-built action from GitHub's marketplace
```

```
- name: Checkout code
  uses: actions/checkout@v4

# Step 2: Configure AWS credentials on the runner machine
# Uses the secrets you added to GitHub — values are never logged
- name: Configure AWS credentials
  uses: aws-actions/configure-aws-credentials@v4
  with:
    aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
    aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
    aws-region: ${{ env.AWS_REGION }}

# Step 3: Authenticate Docker to ECR
# Same command you ran manually — automated here
- name: Login to Amazon ECR
  id: login-ecr
  uses: aws-actions/amazon-ecr-login@v2

# Step 4: Build the Docker image
# github.sha = the git commit hash (e.g. a3f9c2b)
# Using the commit hash as the tag means every image is unique
# and you can always trace exactly which code is in which image
- name: Build Docker image
  env:
    ECR_REGISTRY: ${{ steps.login-ecr.outputs.registry }}
    IMAGE_TAG: ${{ github.sha }}
  run: |
    docker build -t $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG .
    docker tag $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG \
      $ECR_REGISTRY/$ECR_REPOSITORY:latest

# Step 5: Push both tags to ECR
- name: Push image to ECR
  env:
    ECR_REGISTRY: ${{ steps.login-ecr.outputs.registry }}
    IMAGE_TAG: ${{ github.sha }}
  run: |
    docker push $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG
    docker push $ECR_REGISTRY/$ECR_REPOSITORY:latest

# Step 6: Print a summary of what was built
- name: Output image details
  env:
    ECR_REGISTRY: ${{ steps.login-ecr.outputs.registry }}
    IMAGE_TAG: ${{ github.sha }}
  run: |
    echo "Image pushed: $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG"
```

```
    echo "Latest tag:   $ECR_REGISTRY/$ECR_REPOSITORY:latest"
```

3.5 Understanding the Workflow Piece by Piece

What is github.sha?

Every git commit has a unique identifier called a **commit hash** or SHA — a 40-character string like **a3f9c2b4d1e8f5c6d7e8f9a0b1c2d3e4f5a6b7c8**. GitHub exposes this as **${{ github.sha }}** in workflows.

Using the commit SHA as your Docker image tag means you can always trace exactly which version of code is running in production. If something breaks, you look at which image is deployed, note the SHA, and find that exact commit in Git.

What are pre-built Actions (uses:)?

Lines like **uses: actions/checkout@v4** are **pre-built Actions** from GitHub's marketplace. They're reusable workflow components that someone else wrote and published. Instead of writing 20 lines of bash to authenticate with AWS, you use **aws-actions/configure-aws-credentials@v4** which handles it all.

What does runs-on: ubuntu-latest mean?

Every job runs on a **runner** — a virtual machine that GitHub provides. It starts fresh and empty every time. When a workflow triggers, GitHub:

32. Spins up a fresh Ubuntu virtual machine
33. Checks out your repository code onto it
34. Runs your steps in order
35. Shuts down the VM when done (you pay nothing — free tier includes 2000 minutes/month)

3.6 Triggering and Monitoring the Pipeline

1

Create the workflow file and push to trigger it

```
# Create the directory
mkdir -p .github/workflows

# Create the file (paste the YAML from section 3.4)
touch .github/workflows/ci.yml
```

```
# Add and commit
git add .github/workflows/ci.yml
git commit -m 'ci: add GitHub Actions build and push to ECR pipeline'

# Push to main to trigger the pipeline
git push origin main
```

2

Watch the pipeline run

36. Go to your GitHub repository
37. Click the Actions tab at the top
38. You should see your workflow running (yellow spinning circle = in progress)
39. Click on it to see the live log output step by step
40. Green checkmark = success. Red X = failure (click to see error)

A successful first run takes about 3–5 minutes (Docker build + push). Subsequent runs are faster because the runner caches some layers.

3

Verify the image appeared in ECR

```
# Check via CLI
aws ecr describe-images \
  --repository-name docmind \
  --region ap-south-1

# You should see a new image tagged with your commit SHA
# alongside the v1 and latest tags from earlier
```

Or check the AWS Console → ECR → docmind → you should see the new image tagged with the SHA.

3.7 What Happens if the Pipeline Fails?

Error in pipeline	What it means	Fix
Error: No such file: requirements.txt	requirements.txt missing from repo	Add it: pip freeze > requirements.txt, commit
Error: denied: Your Authorization Token has expired	ECR token expired (12hr limit)	Re-run the aws ecr get-login-password command

Error: An error occurred (AccessDenied)	IAM user lacks permission	Add the required policy to your IAM user
Error: invalid argument for --secret	Secret name typo in workflow	Check exact names: AWS_ACCESS_KEY_ID etc.
Build took 10+ minutes	No layer caching on runner	Normal for first run. Consider adding Docker layer caching action.
Image not appearing in ECR	Push step failed silently	Check the Push image step logs for specific error

Part 4: Full Implementation Checklist

4.1 Complete Step-by-Step for Today

Follow these in order. Don't skip ahead.

> **Estimated time**
> AWS account setup + IAM: 20 minutes
> AWS CLI install + configure: 10 minutes
> ECR setup + first manual push: 15 minutes
> GitHub Secrets setup: 5 minutes
> Writing + testing CI workflow: 20 minutes
> Watching pipeline succeed and verifying ECR: 10 minutes
> Total: approximately 80 minutes of active work

Phase 1 — AWS Setup (do this first)

Task	Command / Action	Done?
Create AWS free tier account	aws.amazon.com → Create Account	
Create IAM user docmind-dev	Console → IAM → Users → Create	
Attach policies to IAM user	ECR, ECS, S3, CloudWatch full access	
Create access keys	IAM user → Security credentials → Create access key	
Download .csv file	Save immediately — can't retrieve later	
Install AWS CLI	curl + installer (see Part 1.5)	
Configure CLI	aws configure (region: ap-south-1)	
Verify CLI works	aws sts get-caller-identity	

Phase 2 — ECR + First Push

Task	Command	Done?
Create ECR repository	Console or: aws ecr create-repository --repository-name docmind	
Save repository URI	123456789012.dkr.ecr.ap-south-1.amazonaws.com/docmind	

Authenticate Docker to ECR	aws ecr get-login-password ... | docker login ...	
Tag your image	docker tag docmind:v1 <ecr-uri>:v1	
Push to ECR	docker push <ecr-uri>:v1	
Verify in console	ECR → docmind repo → image visible	

Phase 3 — GitHub Actions CI

Task	Action	Done?
Add AWS_ACCESS_KEY_ID secret	GitHub repo → Settings → Secrets	
Add AWS_SECRET_ACCESS_KEY secret	GitHub repo → Settings → Secrets	
Add AWS_ACCOUNT_ID secret	GitHub repo → Settings → Secrets	
Create .github/workflows/ci.yml	mkdir -p .github/workflows && touch ci.yml	
Paste workflow YAML	Full content from Part 3, Section 3.4	
Commit and push to main	git add . && git commit -m 'ci: add pipeline' && git push	
Watch pipeline in Actions tab	GitHub repo → Actions → watch live logs	
Verify image in ECR	aws ecr describe-images --repository-name docmind	

4.2 Interview Answers for Today's Work

Question	Answer
What is ECR and why use it instead of DockerHub?	ECR is AWS's private container registry. Images in ECR are private by default, live inside your AWS network (faster pulls for ECS), and IAM controls access. DockerHub is public and external.
What is a GitHub Actions runner?	A fresh virtual machine GitHub spins up to run your workflow. Starts clean, runs your steps, shuts down. Free tier includes 2000 minutes per month.
Why tag images with the commit SHA?	Every image maps to an exact version of code. If production breaks, you see which SHA is deployed, find that commit in Git, and know exactly what changed. Far better than just 'latest'.

What is the difference between CI and CD?	CI = automatically build and test on every push. CD = automatically deploy passing builds to an environment. Today we built the CI part. CD (auto-deploy to ECS) is wired up after ECS is configured.
How do you keep AWS credentials secure in CI?	GitHub Secrets — encrypted at rest, injected at runtime as environment variables, never visible in logs (shown as ***). Never put credentials in workflow YAML files.
What triggers your pipeline?	A push to the main branch, or manually via the workflow_dispatch trigger. Only main — pushes to feature branches and dev don't trigger a build.

4.3 Concepts Quick Reference

Term	One-line definition
ECR	AWS private Docker image registry. Like DockerHub inside your AWS account.
ECS	AWS service that runs Docker containers at scale.
Fargate	Serverless compute for ECS. No EC2 servers to manage.
IAM	AWS identity system. Controls who can do what. Always use least privilege.
IAM user	A named identity with specific permissions. Use for CLI and GitHub Actions.
Access key	A key pair (ID + secret) used by programs to authenticate with AWS.
Region	A geographic AWS data centre location. ap-south-1 = Mumbai.
GitHub Actions	GitHub's built-in CI/CD system. Workflows defined in YAML.
Workflow	A YAML file defining automated tasks triggered by events.
Job	A group of steps that run on one runner machine.
Runner	A fresh VM GitHub provides to execute a job.
Secret (GitHub)	An encrypted value stored by GitHub, injected into workflows at runtime.
github.sha	The git commit hash of the code that triggered the workflow.
uses:	Imports a pre-built Action from GitHub Marketplace.
workflow_dispatch	A trigger that adds a manual 'Run workflow' button in the GitHub UI.

The End.